СИСТЕМА УПРАВЛЕНИЯ ЗАДАЧАМИ ПО СПОРТИВНОМУ ПРОГРАММИРОВАНИЮ “THE PROBLEM FORGE”.

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.

Содержание.

О сервисе.....2
Задачи, контесты, права доступа.....3
Процесс создания задачи.....3
Разделы редактирования задачи.....4
Раздел Statement.....4
Раздел General.....4
Раздел Solutions.....5
Раздел Tests & Generators.....6
Раздел Validator & Checker.....7
Раздел Package.....7
Раздел Invocation.....8

О сервисе.

Сервисы для создания задач по спортивному программированию позволяют полностью автоматизировать процесс подготовки задач — от написания условия до построения набора тестов, эталонных решений и итогового пакета для тестирующей системы. Они дают авторам единый интерфейс, в котором можно описать условие, загрузить решения, сгенерировать входные данные и автоматически получить корректный эталонный вывод для каждого теста. Такой подход исключает ошибки, связанные с несогласованностью форматов, ручными опечатками или несовпадением между тестами и правильными ответами. Платформа также предоставляет инструменты для работы с корректными, неправильными и медленными решениями: правильные должны проходить все тесты, неправильные — получать WA, медленные — не укладываться в ограничения. При этом сервис автоматически генерирует большие и разнообразные наборы тестов, проверяет входные данные через валидаторы и использует чекеры для корректного сравнения ответов, что позволяет поддерживать тесты с множеством допустимых решений или с нетривиальным форматом вывода. Это делает проверку более гибкой и справедливой.

Кроме того, сервисы автоматизируют создание примеров, отображаемых в условии: вместо ручного расчёта ответов система запускает основное корректное решение, что исключает человеческие ошибки и повышает точность условия. Такие платформы позволяют авторам запускать решения прямо внутри системы, наблюдать время выполнения, потребление памяти, причины ошибок и корректировать задачу до публикации. Единый формализованный процесс сборки пакета гарантирует детерминированность, воспроизводимость и структурированность — это особенно важно для подготовки целых контестов, где требуется единообразие и отсутствие расхождений между задачами.

Современные олимпиады и соревнования требуют регулярного создания новых задач, а работа над ними часто включает множество внешних инструментов для редактирования, тестирования и обмена материалами, что приводит к дублированию усилий и риску несогласованности данных. Программное средство The Problem Forge решает эту проблему, предоставляя централизованную систему для автоматизации создания, редактирования, тестирования и управления задачами для соревнований по спортивному программированию. Оно интегрируется с сервисом администрирования олимпиад NSUts, что позволяет авторам напрямую публиковать задачи и работать по единым стандартам, снижая вероятность ошибок и ускоряя подготовку соревнований. Таким образом, современные сервисы для создания задач упрощают и ускоряют работу автора, повышают качество финальных задач и обеспечивают стабильную и прозрачную работу тестирующих платформ.

Задачи, контесты, права доступа.

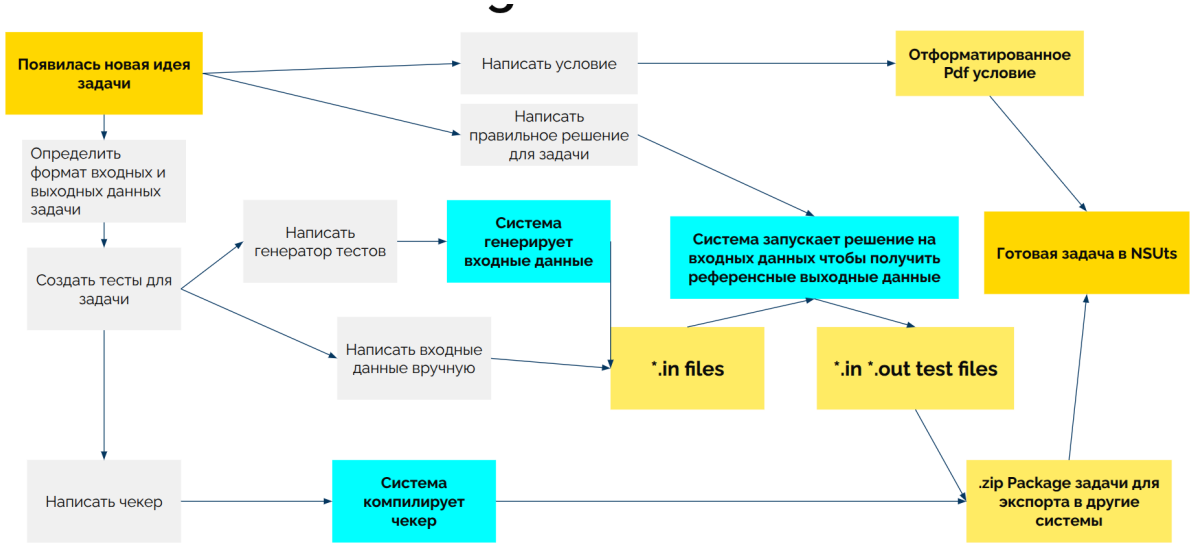
В системе пользователи могут создавать отдельные задачи, а также объединять их в контесты.

Уровни прав доступа подразделяются на три категории: право на изменение, право на чтение и право владения. Владельцем задачи или контеста может быть только один пользователь; право владения является исключительным и не может быть передано другим лицам.

Права доступа назначаются отдельно для задач и для контестов. При предоставлении пользователю прав на доступ к контесту он автоматически получает права на чтение всех задач, входящих в данный контест. Аналогично, при добавлении задачи в контест все пользователи, обладающие правами на чтение этого контеста, получают права на чтение указанной задачи.

Если одна и та же задача включена в несколько контестов, она существует в них в единственном экземпляре. Таким образом, любые изменения, внесённые в задачу, отображаются во всех контестах, в которых она используется.

Процесс создания задачи.



Далее описан процесс создания задачи по пунктам. Для более подробной информации по каждому пункту можно обратиться в документацию по соответствующим разделам редактирования задачи.

1. Определите идею задачи — сформулируйте, что нужно решить, какой алгоритмический приём должен применяться, и оцените ожидаемую сложность, чтобы понимать будущие ограничения.

2. Продумайте модель ввода и вывода — решите, будет один тест или несколько, определите порядок данных, типы переменных и точный формат вывода, чтобы исключить двусмысленность. Раздел Statement.

3. Напишите полное условие задачи — составьте понятный текст с описанием задачи, используйте LaTeX для формул, подробно укажите формат входа и выхода, ограничения на параметры и поясните неочевидные детали. Раздел Statement.

4. Подготовьте граничные и демонстрационные примеры — создайте несколько небольших примеров вручную, добавьте их в тесты и отметьте для показа в условии; подготовьте короткие пояснения, почему ответы именно такие. Раздел Tests + Statement.

5. Разработайте и реализуйте основное корректное решение — напишите надёжный код, учитывающий все особые случаи, добавьте комментарии, проверьте сложность и убедитесь, что он стабилен и укладывается в TL/ML. Раздел Solutions.

6. Реализуйте дополнительные решения — создайте альтернативные корректные версии, а также неправильные и медленные варианты, чтобы проверить силу тестов и правильность выставленных ограничений. Раздел Solutions.

7. Сформируйте тестовый набор вручную — добавьте простые случаи, граничные значения, редкие и сложные конфигурации; проверьте, что они покрывают все потенциальные ошибки, которые может допустить участник. Раздел Tests.

8. Создайте генераторы тестов — напишите программы, которые автоматически генерируют большие и разнообразные входные данные. Раздел Tests.

9. Напишите валидатор и, при необходимости, чекер — валидатор должен гарантировать корректность формата входов, чекер — сравнивать ответы не просто через diff, а с учётом разрешённых отклонений или множества правильных решений. Раздел Validator, Checker.

10. Сгенерируйте полный набор тестов и получите эталонные ответы — запустите основное корректное решение на всех тестах, используйте его вывод как эталонный, убедитесь, что нет ошибок и несовпадений. Раздел Invocation.

11. Прогоните альтернативные решения — проверьте, что правильные решения проходят, неправильные дают WA, а медленные не укладываются в TL, при необходимости укрепите тесты. Раздел Invocation, Tests.

12. Перепроверьте качество условия — убедитесь, что текст однозначен, понятен и не содержит логических дыр; убедитесь, что примеры корректны и совпадают с эталонными выводами. Раздел Statement.

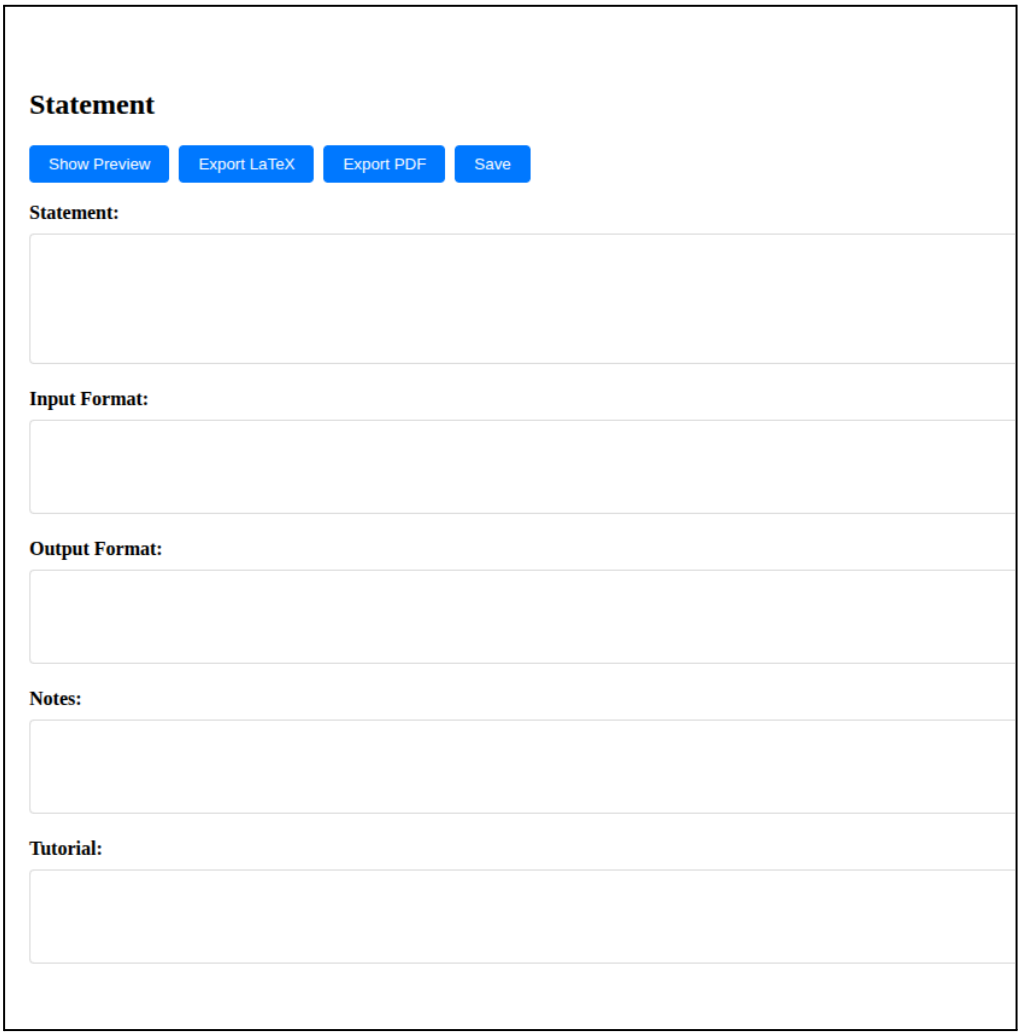
13. Соберите единый пакет задачи — включите текст условия, тесты, эталонные ответы, решения, валидатор, чекер и генераторы; проверьте структуру пакета и воспроизводимость сборки. Раздел Package/Export.

14. Опубликуйте задачу и контролируйте обратную связь — при необходимости исправляйте найденные участниками неточности, обновляйте тесты и следите за качеством задачи после публикации. Раздел Package/Export.

Разделы редактирования задачи.

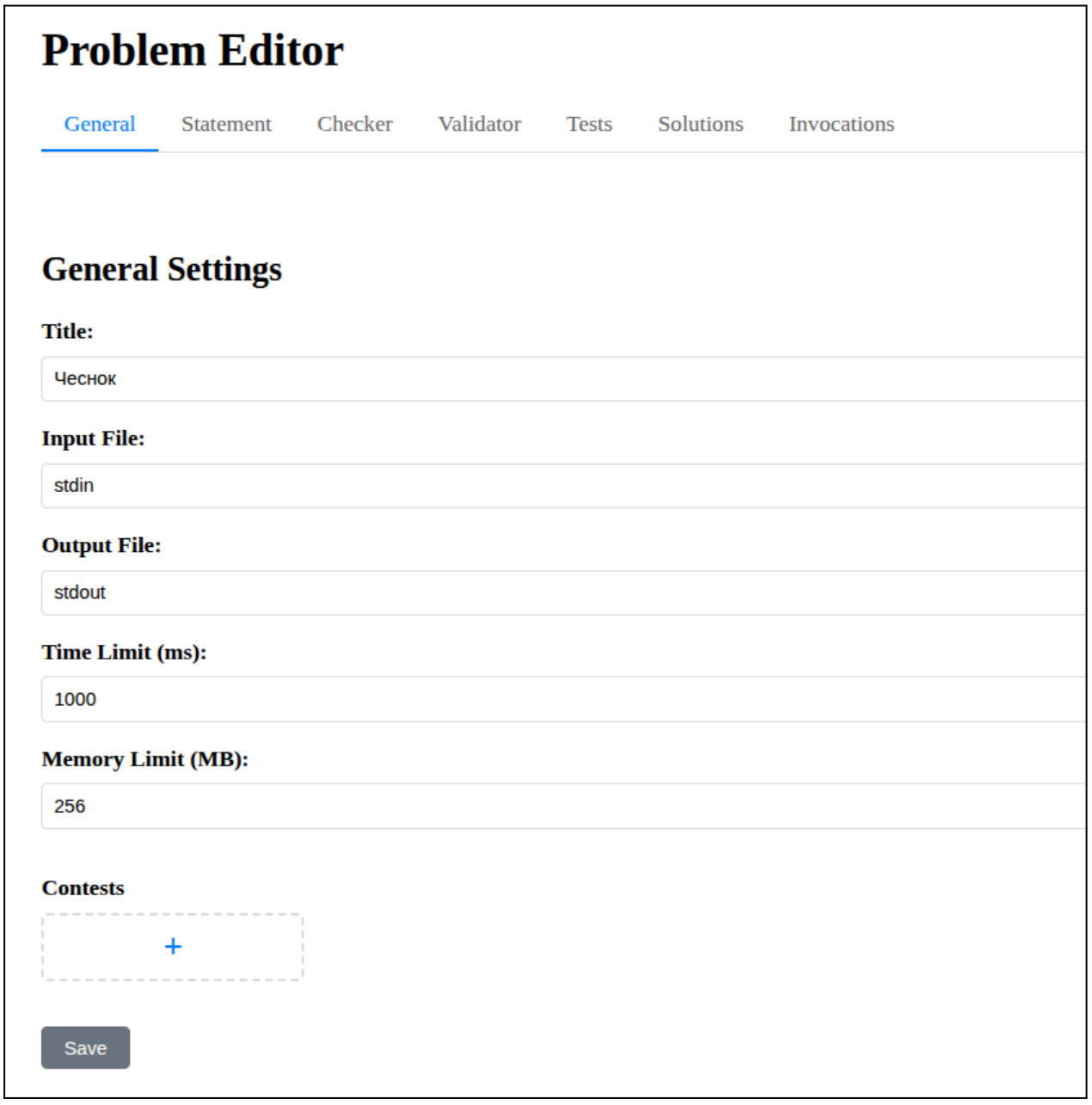
Раздел Statement.

- Составьте полное текстовое условие: ввод, вывод, ограничения, пояснения. Текст условия поддерживает формат Latex формул.
- Определите формат входных и выходных данных
- Чётко пропишите формат входа (строки, числа, количество строк/запросов) и формат вывода.
- Чтобы в условии появились примеры входных и выходных данных вам требуется добавить тесты в разделе “Tests” и отметить чекбокс “show in statement”. Соответственно, для генерации референсных выходных данных также необходимо написать “Main correct solution”. Смотри дальше если пока не понятно.



Раздел General.

Здесь вам нужно выставить ограничения задачи по максимально допустимому потреблению ресурсов - памяти и времени. Классические ограничения для задач - не более 256 Мб памяти и не более 1-4 секунд ограничения по времени. В редких случаях ограничения могут доходить до 1 Гб и 10 секунд.



Раздел Solutions.

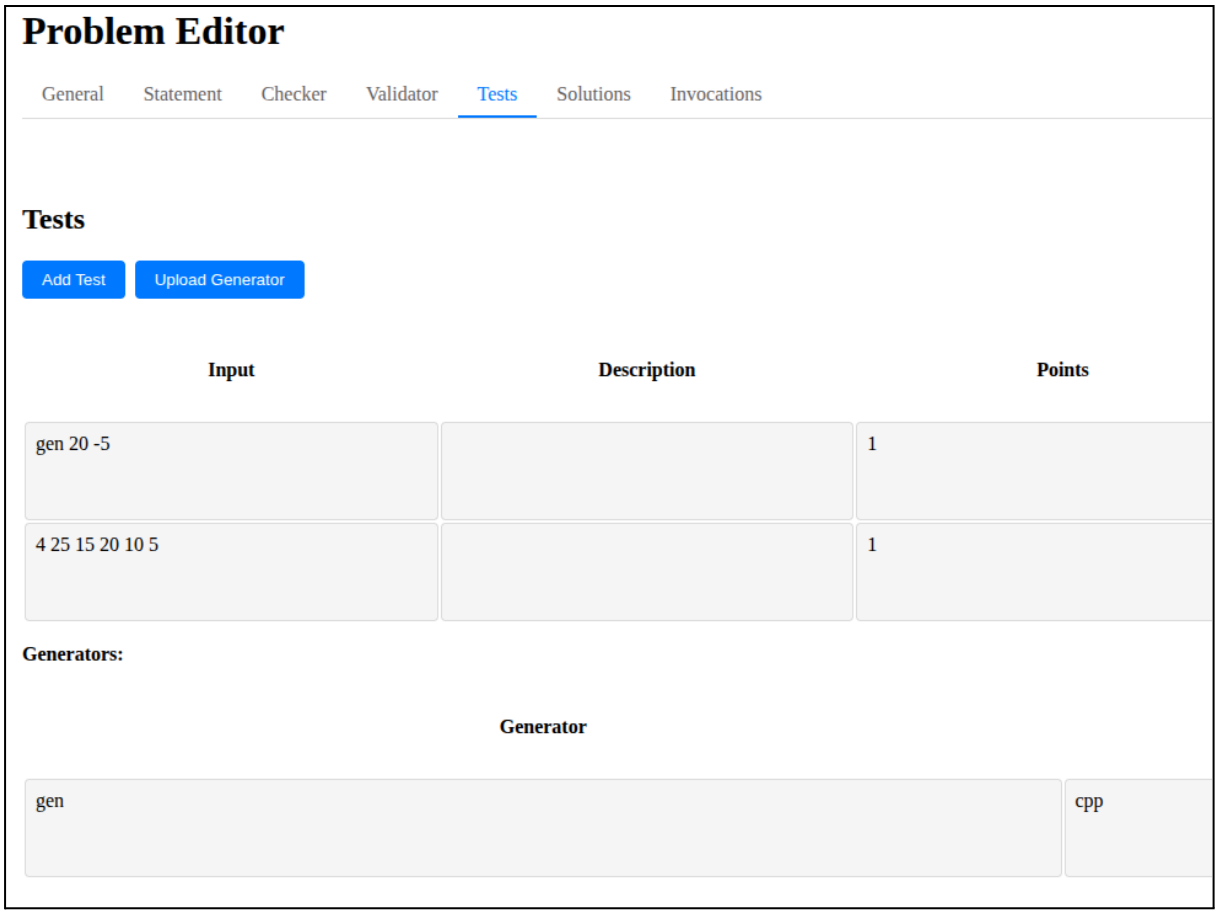
- В этом разделе вы должны написать как минимум одно правильное решение программы. Оно будет использовано для генерации референсных ответов на тест. Также это решение должно укладываться в указанные выше ограничения по потреблению ресурсов.
- Помимо референсного решения рекомендуется также написать несколько других решений: правильных, чтобы проверить, что задача может быть решена разными способами или на разных языках программирования; неправильные, чтобы убедиться, что тесты в задаче достаточно сильные, чтобы обнаружить неверное решение; долгие, чтобы убедиться, что вы верно выставили ограничения по времени и не оптимальные решения не получают вердикт OK.
- У каждого решения в этом разделе есть теги. Если решение помечено, как Main Correct Solution - оно будет использовано для генерации тестов. Все остальные теги несут только декоративную функцию.



Раздел Tests & Generators.

Есть два подхода написания тестов - написание скрипта (генератора), ручной ввод теста.

- В первом случае вам нужно написать скрипт, который будет выдавать входные тестовые данные. Для генерации различных тестов одним генератором используйте аргументы командной строки. Генераторы крайне рекомендуется писать с использованием библиотеки “testlib.h”, которую можно найти по ссылке.
- Во втором случае все очень просто, вы просто вводите входные данные в предназначенное для этого окно. Это полезно для создания сэмплов (тесты, которые будут показаны в условии), написания тестов на граничные случаи.



При запуске тестирования под капотом происходит следующее:

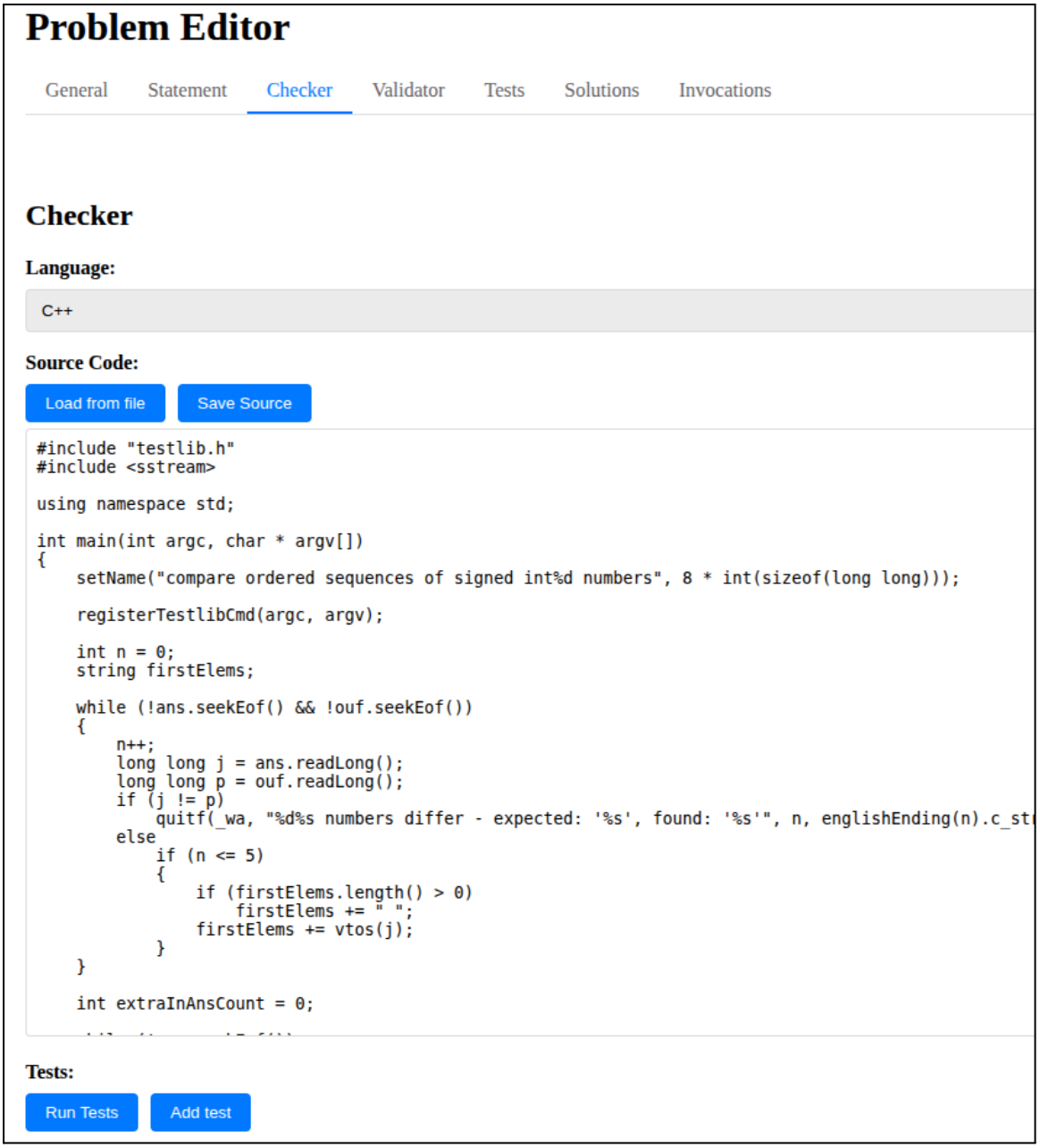
1) Система генерирует тесты генераторного типа.
2) Система запускает валидатор (если он есть) на всех сгенерированных тестах.
3) На тестах, которые оказались валидными запускается решение, помеченное тегом “Main correct solution”.
4) Вывод этого решения на каждом тесте становится референсным ответом на тест.

Раздел Validator & Checker.

В этих разделах вам предлагается написать валидатор и чекер.

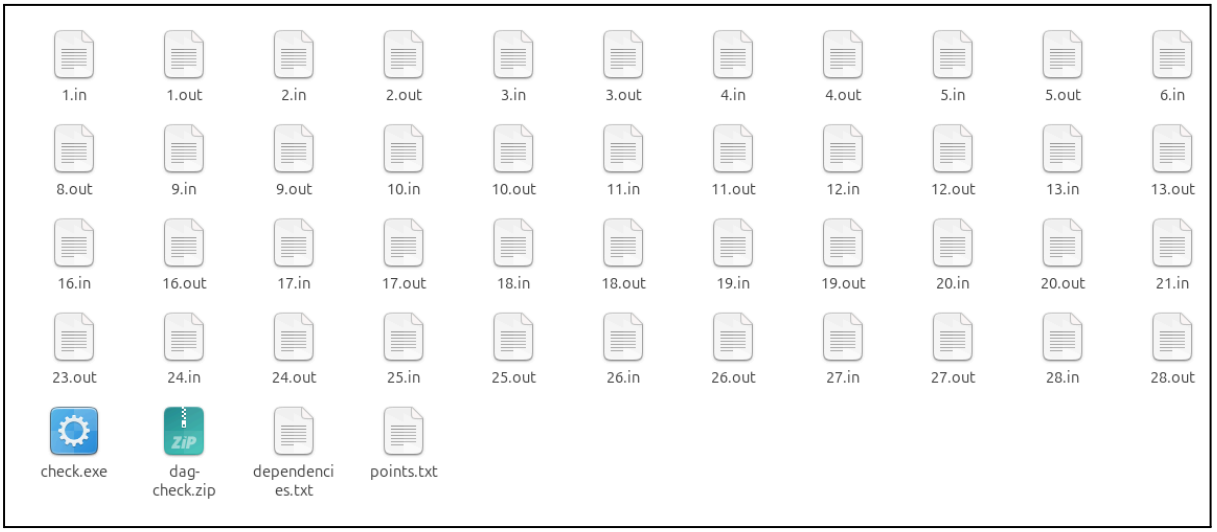
- Чекер нужно написать, если вы хотите, чтобы ответ участника проверялся не обычной командой “diff”, а имел какую-то логику или был нечувствительным к лишним табуляциям в выводе участника.
- Валидатор нужно написать, чтобы он проверил все входные данные тестов, на соответствие заявленным в условии ограничениям.

Эти компоненты задачи, как и генератор рекомендуется писать с помощью библиотеки “testlib.h”, документация по которой может быть найдена по ссылке.



Раздел Package.

В этом разделе сервис автоматически формирует пакет, который можно загрузить в тестирующую систему. Он содержит файлы входных и выходных тестовых данных, чекер, баллы, выставляемые за каждый тест, и зависимости групп. Все это упаковывается в .zip архив.



Раздел Invocation.

В этом разделе вы можете запускать свои решения на тестах, чтобы увидеть вердикты на своем тестовом наборе. Также после запуска будет видна дополнительная информация по каждому тесту.